

Ordem de Criação versus Ordem de Término de Processos Filhos: Estudo Experimental com `fork()` e `wait()`

Fábio Vinnicius Silva Sousa, Jonatas Andrade do Nascimento
Curso de Ciência da Computação, Instituto Federal Campus Maracanaú

Abstract—Um processo pai que cria múltiplos processos filhos por meio de chamadas sequenciais a `fork()` pode dar a falsa impressão de que esses filhos também serão executados e terminarão na mesma ordem em que foram criados. Este trabalho investiga experimentalmente essa hipótese. Foi implementado um programa em C que cria N processos filhos, cada um executando uma carga de trabalho de duração aleatória e independente, e mede a ordem em que o processo pai os recebe de volta via `wait()`. Os experimentos foram repetidos centenas de vezes para diferentes valores de N e comparados com o modelo teórico de uma permutação aleatória uniforme. Os resultados mostram que a hipótese deve ser rejeitada: a probabilidade de a ordem de término coincidir com a ordem de criação cai rapidamente conforme N aumenta, de forma consistente com $1/N!$, evidenciando que o escalonador do sistema operacional não garante nenhuma relação entre ordem de criação e ordem de término dos processos.

Index Terms—sistemas operacionais, `fork`, `wait`, `waitpid`, escalonamento de processos, chamadas de sistema

I. INTRODUÇÃO

Em sistemas operacionais do tipo Unix, um processo pai pode criar múltiplos processos filhos por meio de chamadas sucessivas a `fork()`. Como os filhos são criados sequencialmente, um raciocínio ingênuo poderia supor que essa ordem se refletiria também na ordem em que os filhos terminam sua execução e, consequentemente, na ordem em que o processo pai os recebe de volta via `wait()`.

Problema. A ordem de criação dos processos filhos garante a ordem em que eles serão executados e terminados?

Hipótese. A ordem em que os processos filhos são criados influencia a ordem em que eles serão executados e terminados; consequentemente, ao utilizar `wait()`, o processo pai tenderá a receber os filhos na mesma ordem em que foram criados.

Esta hipótese é razoável à primeira vista, pois os filhos são de fato criados em sequência e herdam código idêntico do pai. Entretanto, uma vez criado, cada processo filho passa a ser uma entidade independente, sujeita ao escalonador do sistema operacional e à duração do trabalho que executa, fatores que não têm relação necessária com a ordem de criação. Este trabalho projeta e executa um experimento controlado para testar essa hipótese.

II. METODOLOGIA

A. Desenho do experimento

Foi implementado um programa em C (`experiment.c`) que, para um parâmetro N informado na linha de comando:

- 1) cria N processos filhos em laço sequencial usando `fork()`, armazenando o PID de cada filho na ordem em que foi criado;
- 2) cada filho executa uma carga de trabalho de duração aleatória e independente, sorteada uniformemente no intervalo $[10\text{ ms}, 210\text{ ms}]$ via `usleep()`, e então encerra repassando seu índice de criação como código de saída (`_exit(i)`);
- 3) o processo pai aguarda os N filhos com `wait()` genérico (sem especificar qual PID esperar), registrando a ordem real em que os filhos terminam;
- 4) ao final, o programa compara a sequência de PIDs de término com a sequência de PIDs de criação e reporta se a ordem foi preservada.

B. Cuidado metodológico: independência das amostras

Uma armadilha identificada durante a implementação merece registro. Em uma primeira versão, o gerador de números pseudoaleatórios era semeado (`srand()`) uma única vez pelo processo pai, antes do laço de `fork()`. Como o `fork()` duplica a memória do processo (*copy-on-write*), todos os filhos herdavam o *mesmo* estado do gerador e, portanto, sorteavam exatamente o mesmo tempo de trabalho, o que invalidava a premissa de amostras independentes e enviesava artificialmente o resultado a favor da hipótese (a ordem de término passava a depender apenas da pequena vantagem de tempo de quem foi criado primeiro). A correção foi semear o gerador *dentro* de cada filho, após o `fork()`, combinando `clock_gettime(CLOCK_REALTIME)` com o PID do próprio filho (Listagem 1). Esse ponto ilustra como um detalhe de implementação pode comprometer silenciosamente a validade de um experimento.

Listing 1. Semeadura independente por filho.

```
if (pid == 0) {
    struct timespec ts;
    clock_gettime(CLOCK_REALTIME, &ts);
    srand((unsigned int)ts.tv_nsec ^ getpid());
    int tempo_us = (rand() % 200000) + 10000;
    usleep(tempo_us);
    _exit(i);
}
```

C. Procedimento estatístico

Para cada valor de $N \in \{2, 3, 4, 5, 10\}$, o programa foi executado repetidamente (`script run_experiments.sh`), e

foi calculada a fração de execuções em que a ordem de término coincidiu exatamente com a ordem de criação. Esse valor observado foi comparado ao valor esperado sob a hipótese nula de que a ordem de término é uma permutação uniformemente aleatória das $N!$ permutações possíveis, isto é, probabilidade $1/N!$ de coincidência exata por acaso.

III. RESULTADOS

A Tabela I apresenta, para cada N , o número de rodadas executadas, a fração de rodadas em que a ordem de criação foi preservada e o valor teórico $1/N!$ esperado sob aleatoriedade uniforme.

TABLE I
FRAÇÃO DE EXECUÇÕES EM QUE A ORDEM DE TÉRMINO COINCIDIU COM A ORDEM DE CRIAÇÃO

N	Rodadas	Ordem preservada	Observado	Esperado ($1/N!$)
2	150	85	56,7%	50,0%
3	150	18	12,0%	16,7%
4	150	10	6,7%	4,2%
5	150	0	0,0%	0,8%
10	100	0	0,0%	$\approx 0,00003\%$

Os valores observados acompanham de perto a ordem de grandeza prevista pelo modelo de permutação aleatória, especialmente para N pequeno. Para $N \geq 5$, a probabilidade teórica já é tão baixa que nenhuma das rodadas executadas preservou a ordem original, como esperado. O padrão dominante é a queda acentuada, praticamente fatorial, da taxa de preservação de ordem à medida que o número de filhos cresce.

Nota: os valores acima foram obtidos no ambiente de desenvolvimento usado para este rascunho. Eles devem ser **regenerados na máquina da equipe** antes da entrega, pois dependem do número de núcleos, da carga do sistema e do escalonador do SO utilizado.

IV. CONCLUSÕES

Os resultados experimentais rejeitam a hipótese original: a ordem de criação dos processos filhos *não* determina a ordem em que eles terminam nem a ordem em que `wait()` os retorna ao processo pai. Um `wait()` genérico simplesmente devolve o próximo filho que terminar, independente de quando foi criado, e a probabilidade de a ordem coincidir com a criação converge rapidamente para a de uma permutação aleatória conforme N cresce. Na prática, isso significa que qualquer código que dependa implicitamente da ordem de criação dos filhos (por exemplo, para associar resultados a tarefas específicas) é incorreto e deve, em vez disso, usar mecanismos explícitos de identificação, como o valor de retorno de `fork()` armazenado pelo pai combinado a `waitpid()` direcionado a um PID específico, quando a ordem de atendimento precisar ser controlada.

REFERENCES

- [1] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ, USA: Wiley, 2018.
- [2] W. R. Stevens and S. A. Rago, *Advanced Programming in the UNIX Environment*, 3rd ed. Boston, MA, USA: Addison-Wesley, 2013.
- [3] Linux manual page, `fork(2)`. [Online]. Available: <https://man7.org/linux/man-pages/man2/fork.2.html>
- [4] Linux manual page, `wait(2)`. [Online]. Available: <https://man7.org/linux/man-pages/man2/wait.2.html>